

Handout: 725103 Data Structure and Algorithm

สัปดาห์ที่ 5: Arrays and Binary Search

ภาคปลาย ปีการศึกษา 2568 | อาจารย์ผู้สอน: ปพนธีร์ แยมโชติ

เป้าหมายการเรียนรู้ประจำสัปดาห์ (Learning Outcomes)

เมื่อจบคาบนี้ นักศึกษาสามารถ...

- เข้าใจวิธีการเก็บข้อมูลใน memory แบบ arrays และคำนวณเลข address ของสมาชิกใน array ได้
- เข้าใจวิธีการค้นหาสมาชิกใน arrays แบบ binary search

คำศัพท์สำคัญ (Key Terms)

- array, index, contiguous memory
- memory address, base address, element size (word size)
- linear search, binary search, low/mid/high
- sorted, invariant (แนวคิดเงื่อนไขที่ต้องจริงตลอด), $O(1)$, $O(n)$, $O(\log n)$

1 Arrays — เก็บข้อมูลแบบ “เรียงเป็นช่อง ๆ”

Definition 1 (Array). array คือโครงสร้างข้อมูลที่เก็บข้อมูลหลายค่าต่อเนื่องกันใน memory (แบบ contiguous) ทำให้เข้าถึงสมาชิกตำแหน่งที่ต้องการได้เร็วมากด้วย index.

1.1 ภาพรวม: array อยู่ใน memory อย่างไร?

ให้มอง memory เป็นช่อง ๆ ที่มี address กำกับ เช่น 1000, 1004, 1008, ... (ถ้าแต่ละช่องกว้าง 4 bytes)

Visualization: contiguous memory + index

สมมติ A เป็น array ของจำนวนเต็ม (int) และ element size = 4 bytes, base address = 1000

index	0	1	2	3	4
A[index]	12	7	25	9	16
address	1000	1004	1008	1012	1016

1.2 สูตรคำนวณ address ของสมาชิกใน array

สูตรสำคัญ: Address Calculation

กรณี index เริ่มที่ 0 (0-based)

$$\text{addr}(A[i]) = \text{base}(A) + i \times w$$

โดย w คือ **element size** (จำนวน bytes ต่อ 1 สมาชิก)

Example 2 (คำนวณ address แบบ 0-based). กำหนด $\text{base}(A)=2000$, $w = 4$ bytes ต้องการหา $\text{addr}(A[7])$

$$\text{addr}(A[7]) = \underline{\hspace{2cm}}$$

Exercise 3 (ฝึกคำนวณ address (ทำในคาบ)). กำหนด $\text{base}(B)=560$, $w = 8$ bytes (เช่น **double**)

1. $\text{addr}(B[0]) =$

2. $\text{addr}(B[3]) =$

1.3 การเข้าถึงและการค้นหาใน array (ระดับแนวคิด)

Operation + Complexity (ระดับแนวคิด)

Operation	คำอธิบาย	Time
Access $A[i]$	คำนวณ address แล้วอ่านค่าได้ทันที	$O(1)$
Update $A[i]=x$	คำนวณ address แล้วเขียนค่า	$O(1)$
Search (ไม่เรียง)	ไล่ทีละช่อง (linear search)	$O(n)$
Search (เรียงแล้ว)	ใช้ Binary Search ได้	$O(\log n)$

2 Binary Search — ค้นหาเร็วขึ้นด้วยเงื่อนไข “ข้อมูลต้องเรียง”

2.1 แนวคิดหลัก

Binary Search ใช้ได้เมื่อไร?

Binary Search ใช้ได้เมื่อ ข้อมูลถูกเรียง (sorted) ตามลำดับเดียวกัน เช่น น้อย → มาก

- ถ้า array ไม่เรียง: Binary Search ใช้ไม่ได้ (อาจตัดครึ่งผิดฝั่ง)
- ถ้า array เรียงแล้ว: เรา “ตัดครึ่งช่วงค้นหา” ได้เรื่อย ๆ

2.2 Visualization: low/mid/high

ให้กำหนดช่วงที่ “เป็นไปได้” ว่าคำตอบอยู่ในช่วง [low..high] แล้วเลือกตรงกลาง mid

Example 4 (ตัวอย่างข้อมูล). ให้ array เรียงแล้ว:

$$A = [3, 7, 10, 14, 19, 22, 31, 35, 40]$$

ต้องการค้นหา target = 19

2.3 Pseudocode ของ Binary Search

Algorithm 1 BinarySearch (ค้นหาใน array ที่เรียงแล้ว)

Require: sorted array A (0-based), target x

```
1:  $low \leftarrow 0$ 
2:  $high \leftarrow |A| - 1$ 
3: while  $low \leq high$  do
4:    $mid \leftarrow \lfloor (low + high)/2 \rfloor$ 
5:   if  $A[mid] = x$  then
6:     return  $mid$ 
7:   else if  $A[mid] < x$  then
8:      $low \leftarrow mid + 1$ 
9:   else
10:     $high \leftarrow mid - 1$ 
11:   end if
12: end while
13: return NotFound
```

2.4 ตาราง Trace (หัวใจของคาบนี้)

Worksheet: Trace Table (กรอก low/mid/high)

ให้ $A = [3, 7, 10, 14, 19, 22, 31, 35, 40]$ และ $x = 19$

step	low	high	mid	$A[mid]$	ตัดสินใจ
1					
2					
3					

2.5 จุดพลาดที่พบบ่อย (ต้องเตือนในคาบ)

Common Pitfalls (เช็คตัวเอง)

- ลืมเงื่อนไข sorted: ข้อมูลไม่เรียงแล้วทำ Binary Search ☐ คำตอบผิด
- เงื่อนไขวนลูป: ต้องใช้ $while (low \leq high)$ ไม่ใช่ $low < high$ (ไม่นั้นอาจพลาดช่องสุดท้าย)
- อัปเดตขอบเขตผิด: เมื่อ $A[mid] < x$ ต้องทำ $low = mid + 1$ (ไม่ใช่ $low = mid$)
- สับสน 0-based/1-based: index ในโจทย์กับใน array ไม่ตรงกัน ☐ off-by-one

2.6 Complexity (ระดับแนวคิด)

ทำไม Binary Search ถึงเร็ว?

ทุกครั้งที่เทียบ เราตัดช่วงค้นหาลงครึ่งหนึ่ง:

$$n \rightarrow \frac{n}{2} \rightarrow \frac{n}{4} \rightarrow \dots$$

จำนวนครั้งที่ตัดจนเหลือ 1 ช่อง $\approx \log_2 n$ ดังนั้นเวลาเป็น $O(\log n)$

Exercise 5 (Mini-Quiz (เช็คความเข้าใจท้ายคาบ)). 1. ถ้า $n = 64$ ต้องเทียบมากที่สุดกี่ครั้งโดยประมาณ? (ใช้ $\log_2 n$)

2. ระหว่าง linear search กับ binary search แบบไหนใช้ได้กับ “ข้อมูลไม่เรียง”?

Checklist ตอนสรุป (1 นาที)

นักศึกษาควรตอบได้ว่า...

- array อยู่ใน memory แบบ **contiguous** จึง access ได้ $O(1)$
- Binary Search ต้องใช้กับข้อมูล **sorted** เท่านั้น
- Trace Table: low/mid/high เปลี่ยนอย่างไรเมื่อ $A[mid] < x$ และเมื่อ $A[mid] > x$

3 การบ้านหลังเรียน

HW1: Address Calculation (ส่งเป็นรูป/ไฟล์ PDF ได้)

กำหนด array C เป็น int ($w = 4$ bytes), $\text{base}(C)=1200$

1. จงหา $\text{addr}(C[5])$

2. (Bonus) ถ้าเป็น 2D array ขนาด 3×4 (row-major), $\text{addr}(C[2][3])$ คือเท่าไร?

HW2: Trace Binary Search 3 เคส (เจอ/ไม่เจอ/ค่าซ้ำ)

ให้ $A = [2, 5, 7, 9, 12, 15, 18, 21, 25, 30]$ (0-based)

(ก) เคสเจอ: $x = 18$ เติมตารางให้ครบจนพบ

step	low	high	mid	$A[mid]$	decision
1					
2					
3					

(ข) เคสไม่เจอ: $x = 8$ เติมตารางให้ครบจนจบรูป (สรุปว่า NotFound)

step	low	high	mid	$A[mid]$	decision
1					
2					
3					
4					

(ค) เคสค่าซ้ำ: ให้ $B = [1, 3, 3, 3, 5, 7, 9]$ และ $x = 3$

1. Binary Search แบบเดิม “มีโอกาส” คืน index ไหนบ้าง? (ตอบได้มากกว่า 1 ค่า)

2. ถ้าอยากได้ ตำแหน่งแรก ของ 3 ต้องทำอะไรเพิ่ม? (อธิบายแนวคิด ไม่ต้องเขียนโค้ด)
